NORTH PHARMACY · QUALITY ASSURANCE

Load and Performance Test Plan

North Pharmacy e-commerce platform

A complete plan for load, stress, spike, soak and breakpoint testing of the North Pharmacy API — built on a measured baseline rather than assumed numbers. The environment was profiled before this plan was written: it saturates at roughly **190 requests per second**, with the knee of the latency curve at about **10 concurrent users**. Every threshold, scenario and script here derives from that measurement.

DOCUMENT	NP-LT-001
VERSION	1.0
STATUS	Issued for review
DATE	25 October 2026
ENVIRONMENT	https://202.4.101.118:1165
TOOLING	k6 (primary), Playwright (correctness gate)
COMPANION TO	NP-API-TS-001 v1.0 — API documentation and test specification
PREPARED BY	QA Automation — devteam@emslbd.com

Contents

1 Introduction

Purpose · Scope · Why now · Audience · Definitions

2 System under test

Architecture · What the API costs · Properties that shape the plan

3 Measured baseline

Per-endpoint latency · The concurrency ramp · What the numbers mean · Caveats

4 Workload model

User populations · Journeys · Request mix · Converting users to load

5 Performance objectives

Service level objectives · How each was chosen · Pass and fail

6 Test types and scenarios

Smoke · Load · Stress · Spike · Soak · Breakpoint · Targeted scenarios

7 Blockers that must be cleared first

One shared account · Rate limiting · Write isolation

8 Implementation

k6 project · Shared module · Full scripts · Running · CI

9 Monitoring and analysis

Client metrics · Server metrics · Reading a result · Common misreadings

10 Tuning candidates

Five changes, ranked by expected effect per unit of work

11 Execution, risks and reporting

Schedule · Entry and exit criteria · Risks · Report template

12 Appendices

1 Introduction

1.1 Purpose

This plan defines how the North Pharmacy platform is tested for performance under concurrent use: what is measured, against what targets, with which tools, and how a result is judged. It covers the API rather than the browser, because that is where capacity is actually consumed — a browser test measures rendering as well, needs far more hardware per simulated user, and cannot reach the request rates that matter.

1.2 Scope

IN SCOPE	OUT OF SCOPE
Throughput and latency of the public catalogue and search endpoints	Browser rendering, paint time, Core Web Vitals
Authenticated customer journeys: cart, account, orders	Static asset delivery and CDN behaviour
Administration reads under concurrent operator use	Database tuning and index design
Behaviour at and beyond saturation — degradation mode	Network capacity planning between Dhaka and the host
Endurance over a working day	Disaster recovery and failover
Rate limiting under sustained pressure	Functional correctness — NP-API-TS-001 owns that

1.3 Why this plan exists now

Three things make performance testing worth doing on this product at this point.

1. **The catalogue is large.** 17,453 products with faceted search. The query that drives the shop page is already five to fifteen times more expensive than any other endpoint, and it is the one every customer hits first.
2. **The audience is on mobile data.** NP-QA-TC-001 names mobile as the dominant channel. Responses are currently uncompressed, so a 17 KB JSON page is 17 KB on the wire. Latency on a 3G connection is dominated by bytes, not by server time.
3. **Nothing has ever been measured.** Before this plan there was no baseline at all, so there was no way to tell a regression from normal variation. Section 3 establishes that baseline.

1.4 Audience

- **QA engineers** running the tests — sections 6, 8 and 9.
- **Backend developers** acting on results — sections 3, 9 and 10.
- **Infrastructure** sizing the production host — sections 3, 4 and 5.
- **Product and release management** — sections 5 and 11 carry the targets and the gates.

1.5 Definitions

TERM	MEANING IN THIS DOCUMENT
VU	Virtual user — one simulated client executing a journey in a loop

TERM	MEANING IN THIS DOCUMENT
RPS	HTTP requests completed per second, measured at the client
p95 / p99	The latency below which 95% / 99% of requests complete
Knee	The concurrency at which added load stops increasing throughput and starts increasing latency instead
Saturation	The state beyond the knee: throughput flat, latency rising in proportion to offered load
Think time	Simulated pause between a user's actions. Omitting it is the most common way to make a load test meaningless
Error rate	Share of requests that are not 2xx, *or* that fail a correctness check

2 System under test

2.1 Architecture

```
Browser (React SPA)
  |   same-origin fetch, X-Requested-With: NorthPharmacy
  v
IIS  ->  ASP.NET Core API  ->  SQL Server
         /api/*                17,453 products, 50 orders, 7 prescriptions
         /uploads/*            product photos and manufacturer logos
```

A single host serves static assets and the API. Session state is a Data Protection cookie, which means authentication is cryptographic rather than a session-store lookup — it costs CPU per request but no shared state, so it scales horizontally without sticky sessions.

2.2 Properties that shape this plan

PROPERTY	OBSERVED	CONSEQUENCE FOR LOAD TESTING
Protocol	HTTP/1.1, no HTTP/2	Six-connection-per-origin browser limit applies. A load client must model connection reuse, not open unlimited sockets
Compression	Not negotiated	Bytes on the wire equal bytes of JSON. Payload size is a first-class metric here, not a footnote
TLS	Self-signed	The client must skip verification. TLS handshake cost is real — keep connections alive or it dominates
Guard header	`X-Requested-With: NorthPharmacy`	**Every request needs it**, or the server answers 400 without doing any work — producing a beautiful, meaningless result
Rate limiting	Auth endpoints only	Reads can be driven hard. Authentication cannot — see section 7.2
Accounts	One customer, one admin	**The single biggest blocker.** See section 7.1
Order list	Unpaginated	Cost grows with a customer's history. Scenario LT-S6 targets it

The guard header is the single most common way to get a wrong result

A load script that omits `X-Requested-With: NorthPharmacy` receives `400 {"title":"Request rejected."}` from middleware that runs before any controller, any query and any serialisation. The run will report superb latency and enormous throughput while measuring nothing but the rejection path. Every script in section 8 sets the header centrally, and every scenario asserts on status 200 rather than merely on timing — so this failure mode shows up as a 100% error rate instead of as a flattering number.

3 Measured baseline

The environment was profiled on 25 October 2026 before this plan was written, so that every threshold below rests on an observation rather than a guess. Two measurements were taken: warm single-request latency per endpoint, and a concurrency ramp over a mixed read workload.

3.1 Single-request latency

Twelve sequential samples per endpoint after one warm-up request, keep-alive on, no concurrency. This is the floor — the cost of the work itself with no queueing.

ENDPOINT	P50	P95	MAX	BYTES	ROLE IN THE WORKLOAD
`/api/catalog/districts`	1 ms	1 ms	1 ms	0.7 KB	Health check
`/api/auth/me`	1 ms	1 ms	1 ms	<0.1 KB	Every page load
`/api/store/config`	2 ms	3 ms	3 ms	0.9 KB	Every page load
`/api/catalog/categories`	2 ms	2 ms	2 ms	1.8 KB	Every page load
`/api/catalog/manufacturers`	2 ms	2 ms	2 ms	10.3 KB	Manufacturer page
`/api/store/content`	3 ms	3 ms	3 ms	7.7 KB	Every page load
`/api/catalog/products/{id}`	3 ms	4 ms	4 ms	12.1 KB	Product detail
`/api/catalog/suggest?q=`	5 ms	7 ms	7 ms	2.0 KB	Fires per keystroke
`/api/catalog/home`	5 ms	6 ms	6 ms	19.3 KB	Home page — largest payload
`/api/catalog/products?q=napa`	7 ms	13 ms	13 ms	16.3 KB	Search results
`/api/catalog/products`	**15 ms**	**17 ms**	17 ms	**17.3 KB**	**Shop page — the hot path**

Uncontended, the API is fast: everything answers in single-digit or low double-digit milliseconds. One endpoint stands apart. The unfiltered catalogue query costs 15 ms against 1–5 ms for everything else, and it is the one a customer hits on the shop page, on every filter change and on every page of results. It is therefore weighted most heavily in the workload model and gets its own scenario in section 6.

3.2 The concurrency ramp

A mixed read workload — weighted roughly as a real browsing session — driven at increasing concurrency, five seconds per step, keep-alive on, no think time.

VUS	REQUESTS	RPS	P50	P95	P99	MAX	ERRORS	MEAN IN FLIGHT
1	628	126	4 ms	15 ms	35 ms	323 ms	0	8 ms
5	890	178	14 ms	133 ms	231 ms	582 ms	0	28 ms
10	**982**	**196**	**20 ms**	**237 ms**	336 ms	1,020 ms	0	**51 ms — the knee**
25	826	165	45 ms	619 ms	1,048 ms	2,558 ms	0	152 ms

VUS	REQUESTS	RPS	P50	P95	P99	MAX	ERRORS	MEAN IN FLIGHT
50	913	183	117 ms	1,095 ms	1,968 ms	2,942 ms	0	273 ms

"Mean in flight" is VUs ÷ RPS — Little's Law — giving the average time a request spent in the system including queueing.

3.3 What the numbers mean

Maximum throughput	**about 190 RPS** for this read mix. Throughput peaks at 196 at 10 VUs and does not improve with more load
Knee of the curve	**about 10 concurrent requests.** Below it, added concurrency buys throughput. Above it, it buys only latency
Degradation mode	**Queueing, not failure.** Zero errors at every level, including 50 VUs. The server slows rather than shedding load
Latency growth	Beyond the knee, mean latency grows almost exactly in proportion to offered load — 51 ms at 10, 152 ms at 25, 273 ms at 50. The textbook signature of a saturated fixed-width resource
Tail behaviour	p99 reaches 1,968 ms and the worst request 2,942 ms at 50 VUs. The tail degrades far faster than the median, so p50 alone would hide the problem entirely

> **Degrading by queueing is the good failure mode**
>
> Many systems under overload start returning 500s, dropping connections or exhausting a thread pool. This one simply queues: 4,239 requests at up to fifty times the comfortable concurrency produced **zero** non-200 responses. That is a solid foundation. It also means monitoring must watch latency, because error rate will not warn anyone — by the time errors appear, the system will have been unusably slow for some time.

3.4 Capacity in customers

190 RPS is only meaningful once converted into people. The conversion depends on how many API calls a page costs and how long users pause between pages, so three honest scenarios are given rather than one confident number.

ASSUMPTION	CALLS/PAGE	THINK TIME	RPS/USER	CONCURRENT USERS AT 190 RPS
Heavy — impatient browsing, search-as-you-type	5	5 s	1.00	**~190**
Typical — browse, read, add to cart	4	12 s	0.33	**~570**
Light — considered shopping, long pauses	3	25 s	0.12	**~1,580**

The planning figure this document adopts is the middle row: **roughly 550–600 concurrent browsing customers** before the platform reaches saturation. For a single-city pharmacy that is comfortable headroom; the point of measuring it is to know where the edge is before a campaign or a public holiday finds it first.

3.5 Caveats — read these before quoting the numbers

> The baseline above is **indicative, not formal**. It is good enough to set thresholds and shape this plan, and it should be re-established with k6 from a dedicated generator before any figure here is quoted outside the team.

1. **Single generator, same network.** The client ran on one machine close to the host, so network latency is near zero — real customers on Bangladeshi mobile data will add tens to hundreds of milliseconds, and far more for 17 KB of uncompressed JSON. These figures measure the *server*, not the user experience.
2. **The client may be part of the ceiling.** A single Node.js process drives all sockets from one thread. At 50 VUs some of the measured latency may be client-side queueing. The 190 RPS plateau is therefore a *lower bound* on server capacity.
3. **Five seconds per step.** Long enough to see the shape, too short to see garbage collection, connection-pool churn or cache warming. Section 6 specifies proper durations.
4. **Reads only.** No writes, no authentication, no database contention from concurrent mutation. Write-path capacity is unmeasured — see section 7.
5. **A shared environment.** Other activity during the window is possible and was not controlled for.

4 Workload model

A load test is only as good as its workload model. Hammering one endpoint measures that endpoint; it does not tell anyone whether the platform works on a busy evening. The model below is built from the actual call sequences captured from the SPA, so the mix reflects what the application really does rather than what seems reasonable.

4.1 Observed page costs

Captured by recording all XHR traffic while walking each screen.

PAGE	CALLS	ENDPOINTS
Home	5	`store/config`, `store/content`, `catalog/categories`, `catalog/home`, `auth/me`
Shop / filter	1	`catalog/products` — one per filter, sort or page change
Search typing	1–4	`catalog/suggest`, debounced per keystroke
Product detail	1	`catalog/products/{id}`
Cart	2	`cart`, `cart/quote`
Sign in	3	`auth/login`, `cart`, `wishlist/ids`
Account	1	`account/summary`
Checkout	1	`checkout/options`
Admin dashboard	2	`admin/summary`, `admin/dashboard`

4.2 User populations

POPULATION	SHARE	AUTHENTICATED	BEHAVIOUR
Anonymous browser	60%	No	Lands on home, browses the shop, filters, opens products. Never signs in
Searcher	20%	No	Arrives with a medicine in mind. Types, uses suggestions, opens a result
Shopper	15%	Yes	Signs in, browses, adds to cart, quotes, reaches checkout
Returning customer	4%	Yes	Signs in, checks orders and account, leaves
Administrator	1%	Yes (admin)	Dashboard, order queue, prescription queue, inventory. Few users, heavy queries

These shares are a reasoned starting model, not measured traffic. Replace them with real figures as soon as production analytics exist — they are isolated in one constant in the k6 code so that a change is a one-line edit.

4.3 Resulting request mix

ENDPOINT	WEIGHT	AT 190 RPS	WHY
`catalog/products`	25%	48 rps	Shop page, every filter, every page
`catalog/products/{id}`	15%	29 rps	Product detail

ENDPOINT	WEIGHT	AT 190 RPS	WHY
`catalog/suggest`	14%	27 rps	Type-ahead — several per search
`store/config`	10%	19 rps	Every page load
`auth/me`	10%	19 rps	Every page load
`catalog/home`	8%	15 rps	Landing
`store/content`	6%	11 rps	Every page load
`catalog/categories`	5%	10 rps	Navigation panel
`cart` and `cart/quote`	4%	8 rps	Authenticated shoppers only
Account, orders, admin	3%	6 rps	Small share, heavier queries

4.4 Think time

Every journey includes randomised think time. Without it a VU is not a user — it is a benchmark loop, and twenty of them will saturate this platform while twenty real customers would not notice each other.

ACTION	THINK TIME
Between page navigations	8–20 s, uniform
Between search keystrokes	150–400 ms
Reading a product page	10–40 s
Filling the checkout form	30–90 s
Administrator between actions	5–15 s

5 Performance objectives

Thresholds are derived from section 3 rather than chosen from habit. Each is set at a level the platform demonstrably meets today, so a breach means something changed — which is the only kind of threshold worth having. All figures are measured at the load client on the same network as the host, so they are server-side and directly comparable to the baseline.

5.1 Service level objectives

OBJECTIVE	TARGET	BASELINE	RATIONALE
p50 latency, whole mix	< 50 ms	20 ms at 10 VU	2.5× headroom over the knee
p95 latency, whole mix	< 400 ms	237 ms at 10 VU	Below the 619 ms seen at 25 VU, so a breach means the knee has moved
p99 latency, whole mix	< 1,000 ms	336 ms at 10 VU	One second is where a page stops feeling responsive
p95, light reads	< 150 ms	1–3 ms solo	These must stay cheap; if they do not, the whole host is contended
p95, `catalog/products`	< 500 ms	17 ms solo	The heaviest query, given its own looser bound
Error rate	< 0.1%	0% at every level	The platform currently never errors under load. Any error is a regression
Throughput, mixed read	≥ 180 RPS	196 RPS peak	Within measurement noise of the observed ceiling
Soak drift	< 20% over 2 h	unmeasured	Catches leaks, pool exhaustion and cache growth
Spike recovery	< 60 s to baseline	unmeasured	A flash crowd must not leave the platform degraded

5.2 Per-endpoint budgets

Each budget is the solo p95 from section 3.1, multiplied by a factor that reflects how much queueing that endpoint should be allowed to absorb at target load.

ENDPOINT	SOLO P95	BUDGET P95	FACTOR	CLASS
`/api/catalog/districts`	1 ms	100 ms	100×	Health check — must never be slow
`/api/auth/me`	1 ms	100 ms	100×	Light
`/api/store/config`	3 ms	150 ms	50×	Light
`/api/catalog/categories`	2 ms	150 ms	75×	Light
`/api/store/content`	3 ms	200 ms	67×	Medium — 7.7 KB
`/api/catalog/products/{id}`	4 ms	300 ms	75×	Medium — 12.1 KB
`/api/catalog/suggest`	7 ms	250 ms	36×	Interactive — must feel instant
`/api/catalog/home`	6 ms	400 ms	67×	Heavy — 19.3 KB

ENDPOINT	SOLO P95	BUDGET P95	FACTOR	CLASS
`/api/catalog/products`	17 ms	500 ms	29×	Heaviest
`/api/admin/dashboard`	unmeasured	1,500 ms	—	Aggregate query, few users, tolerant

> `catalog/suggest` gets the tightest factor on purpose. It fires while someone is typing, so 250 ms is the point at which suggestions stop feeling attached to the keyboard. It is not the most expensive endpoint, but it is the most latency-sensitive one.

5.3 Pass, warn and fail

VERDICT	CONDITION	ACTION
PASS	All SLOs met	Record the run; update the trend
WARN	Any SLO within 20% of its limit, or 10% worse than the previous run	Raise a performance observation; investigate before the next release
FAIL	Any SLO breached, or error rate above 0.1%	Blocks release. Raise a defect with the run artefacts attached

6 Test types and scenarios

Six standard test types plus four targeted scenarios. Each answers a different question, and the value comes from running them as a set rather than picking one.

TYPE	QUESTION IT ANSWERS	SHAPE	DURATION
Smoke	Does the script work and is the environment up?	2 VU flat	1 min
Load	Does the platform meet its SLOs at expected peak?	Ramp to 10 VU, hold	20 min
Stress	What happens beyond the knee, and how badly?	Step to 60 VU	30 min
Spike	Does it survive a flash crowd, and does it recover?	2 → 80 → 2 VU	12 min
Soak	Does it degrade over a working day?	8 VU flat	2–8 h
Breakpoint	Where exactly does it break?	Arrival rate to failure	Until threshold breach

6.1 LT-01 Smoke

Purpose	Prove the script, the guard header, the TLS exception and the environment before spending 20 minutes on a real run
Profile	2 VUs, 1 minute, full mixed journey
Pass	0 errors, p95 under 100 ms, every check green
Run it	Before every other test, and in CI on every build

This is the cheapest guard against the failure described in section 2.2 — a script missing the guard header will show a 100% check failure here in sixty seconds rather than producing a flattering twenty-minute result.

6.2 LT-02 Load — the main event

Purpose	Verify the SLOs of section 5 at expected peak traffic
Profile	Ramp 0→10 VU over 2 min, hold 15 min, ramp down 3 min
Why 10	The measured knee. With think time this represents roughly 550–600 concurrent browsing customers
Mix	Section 4.2 populations in their stated shares
Pass	All section 5.1 SLOs met for the whole hold period, not just on average
Cadence	Every release candidate

6.3 LT-03 Stress

Purpose	Characterise behaviour beyond the knee. Not to pass — to understand

Profile	Steps of 5 minutes at 10, 20, 30, 45 and 60 VU
Record	RPS, p50, p95, p99, error rate and error *type* at each step
Expected	Flat throughput near 190 RPS with latency rising in proportion. Based on the baseline, errors should stay at zero throughout
Pass	No 5xx, no connection resets, and full recovery within 2 minutes of ramp-down

> The useful output of a stress test is not a number but a sentence: *"beyond N concurrent requests the platform degrades by queueing, latency rises linearly, and no requests are lost."* That sentence is what lets operations decide whether to add a queue, a cache or a second host — and it is already half-written from the section 3 baseline.

6.4 LT-04 Spike

Purpose	Model a campaign SMS, a social post or an epidemic news story
Profile	2 VU for 3 min → **80 VU within 30 s** → hold 3 min → drop to 2 VU → observe 5 min
Watch	Time to first error; peak p99; and above all **time to return to baseline latency** after the drop
Pass	Recovery to baseline p95 within 60 s of the drop; no 5xx; no restart required

Recovery matters more than the peak. A platform that is slow for three minutes during a spike is acceptable; one that stays slow for twenty minutes afterwards has a queue it cannot drain, and that is a design problem rather than a capacity one.

6.5 LT-05 Soak

Purpose	Find what only time reveals: memory growth, connection-pool exhaustion, unbounded caches, log-disk pressure
Profile	8 VU — comfortably below the knee — for 2 hours minimum, 8 hours preferred overnight
Compare	p95 over the first 15 minutes against the last 15 minutes
Pass	Drift under 20%; flat server memory; no growth in error rate
Cadence	Monthly, and before any major release

6.6 LT-06 Breakpoint

Purpose	Find the true ceiling with an open model, where arrival rate is fixed regardless of how slow responses become
Profile	`ramping-arrival-rate`, 50 → 600 RPS over 20 minutes, aborting on threshold breach
Why open	A closed VU model self-throttles: slow responses mean fewer requests, which hides the cliff. Real users do not wait their turn
Record	The arrival rate at which p95 crosses 1 s, and the rate at which errors first appear

> LT-06 is the one test that can genuinely disrupt a shared environment, because an open model will happily queue thousands of requests. Run it in an agreed window, never alongside manual testing, and have someone watching the host.

6.7 Targeted scenarios

ID	SCENARIO	PROFILE	WHAT IT PROVES
LT-S1	Catalogue query under pressure	10 VU, 10 min, `catalog/products` only, randomised filters and pages	Isolates the heaviest endpoint. Deep pages and rare filter combinations are where an index is missing
LT-S2	Search-as-you-type storm	15 VU typing 4–6 character terms with 150–400 ms gaps	`suggest` fires per keystroke, so a modest number of searchers produces a high request rate. The most under-estimated load on the system
LT-S3	Signed-in shopper journey	10 VU, login once then browse, cart, quote	Adds cookie decryption and per-user state. **Needs section 7.1 resolved first**
LT-S4	Administrator concurrency	3 VU on dashboard, orders, prescriptions, inventory	Few users, heavy aggregate queries. Proves back-office work does not starve the storefront
LT-S5	Mixed storefront and admin	LT-02 plus LT-S4 together	The realistic case. Reveals contention invisible when each is run alone
LT-S6	Unbounded order list	Seed a customer with 500+ orders, then `GET /api/orders` at 5 VU	Quantifies finding F-5 in NP-API-TS-001. Turns a theoretical concern into a number

7 Blockers that must be cleared first

Three constraints limit what can be tested today. The first makes half this plan impossible until it is fixed, so it is stated plainly rather than buried.

7.1 One shared account per role BLOCKER

> The environment issues **one** QA customer and **one** administrator. Every authenticated scenario — LT-S3, LT-S4, LT-S5, LT-S6 and the authenticated share of LT-02 — needs many distinct users, and cannot run meaningfully until they exist.

Pointing many VUs at one account does not merely risk odd results; it measures the wrong thing in four specific ways:

1. **Cart and wishlist collide.** All VUs mutate one server-side cart, so every assertion about cart contents becomes a race.
2. **Login throttling engages.** Repeated concurrent sign-ins trip the brute-force control that NP-LOG-TC-010 asserts, and the account locks for everyone — including the functional suite and anyone testing by hand.
3. **Per-user caching is wrong.** One user's data in every cache gives an unrealistically high hit rate, so the result is better than reality.
4. **Database contention is wrong.** Real load spreads writes across thousands of rows; one account concentrates them on a handful, which may be better or worse than production but is certainly not the same.

Required: at least **100** seeded customer accounts (`lt_user_001` … `lt_user_100` , distinct mobile numbers, known password) and **5** administrators, with a reseed script. Until then, scenarios LT-01, LT-02 (anonymous share only), LT-03, LT-04, LT-06, LT-S1 and LT-S2 can run — which is the majority of the read workload, and enough to be useful.

7.2 Rate limiting on authentication CONSTRAINT

`POST /api/auth/register` returns 429 from about the tenth request in a minute from one client, and `POST /api/auth/login` carries brute-force throttling. The limiter sends no `Retry-After` , so a client cannot discover the window.

RULE FOR LOAD TESTING	REASON
Each VU signs in **once**, in `setup()` or on first iteration, and reuses the cookie	Mirrors real users and keeps login out of the measured loop
Never include `register` in a load scenario	It is rate limited by design and creates permanent data
Stagger VU start-up over 30–60 s	A hundred simultaneous sign-ins look exactly like an attack
Treat any 429 as a *result*, not an error	Count it separately; it is information about the limiter, not a platform failure

Read endpoints are not rate limited — 30 rapid requests to `/api/catalog/categories` produced zero 429s — so the read workload can be driven as hard as the generator allows.

7.3 Write isolation CONSTRAINT

No load scenario places an order, uploads a prescription, changes a setting or reviews a prescription. Those paths are destructive against shared seed data and are gated in the functional suite for the same reason. Write-path capacity is

therefore **unmeasured**, and that is a known gap rather than an oversight.

`POST /api/cart/quote` is the exception and the opportunity: it performs the full pricing calculation — subtotal, discount, delivery, total — writes nothing, needs no authentication and is not rate limited. It is the closest available proxy for checkout cost, and LT-S3 uses it for exactly that.

Measuring the true write path needs a disposable environment with its own database. That is the single change that would complete this plan, and it is recommended as the next infrastructure step after account seeding.

8 Implementation

8.1 Tool choice

TOOL	ROLE	WHY
k6	Primary load tool	Scripts are JavaScript, so they read like the Playwright suite the team already maintains. Native percentile thresholds, per-request tagging, open and closed workload models, single static binary, no JVM
Playwright	Correctness gate	The functional suite runs before and after each load test to prove nothing broke. Load tools check status codes; they do not check that an order total is right
Grafana / InfluxDB	Optional trend store	Only worth adding once runs are regular. The JSON summary is enough to start

JMeter would also do the job. k6 is chosen because the scripts stay in the same language and repository as the existing automation, which is what makes them likely to be maintained rather than written once and abandoned.

8.2 Project layout

```
load/
  lib/common.js         shared client: guard header, metrics, think time
  smoke.js              LT-01
  load.js               LT-02   ← the main scenario
  stress.js             LT-03
  spike.js              LT-04
  soak.js               LT-05
  breakpoint.js         LT-06
  s1-catalogue.js       LT-S1
  s2-suggest.js         LT-S2
  s3-shopper.js         LT-S3   requires seeded accounts (§7.1)
  s4-admin.js           LT-S4   requires seeded accounts
  results/              JSON summaries, one per run, committed for trend
```

8.3 The shared module

Everything else imports this. The guard header is set in exactly one place, which is the point.

```
// load/lib/common.js
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Rate } from 'k6/metrics';

export const BASE = __ENV.BASE_URL || 'https://202.4.101.118:1165';

/* Required on EVERY request — NP-API-TS-001 §3.3.
   Without it the API answers 400 {"title":"Request rejected."} from middleware,
   before any controller or query runs. The run would look fast and mean nothing. */
export const GUARD = { 'X-Requested-With': 'NorthPharmacy', Accept: 'application/json' };

export const errors   = new Rate('np_errors');
export const rejected = new Rate('np_guard_rejected');   // must stay at 0
export const limited  = new Rate('np_throttled_429');    // information, not failure

function record(res, name) {
  const ok = check(res, {
    [`${name} 200`]:  (r) => r.status === 200,
    [`${name} body`]: (r) => r.body && r.body.length > 0,
  });
  errors.add(!ok);
  rejected.add(res.status === 400 && String(res.body).includes('Request rejected'));
  limited.add(res.status === 429);
  return res;
}

export function get(path, name) {
  return record(http.get(BASE + path, { headers: GUARD, tags: { endpoint: name } }), name);
}

export function post(path, body, name) {
  return record(http.post(BASE + path, JSON.stringify(body), {
    headers: { ...GUARD, 'Content-Type': 'application/json' },
    tags: { endpoint: name },
  }), name);
}

/* A VU without think time is a benchmark loop, not a user. */
export const think = (min, max) => sleep(min + Math.random() * (max - min));
export const pick  = (a) => a[Math.floor(Math.random() * a.length)];

export const SEARCH_TERMS = ['napa', 'paracetamol', 'ace', 'seclo', 'maxpro',
                             'monas', 'fexo', 'amodis', 'sergel', 'napa extra'];

/* Fetch real ids once, so the run exercises the real catalogue rather than
   one hard-coded product that may be cached warm. */
export function catalogueSeed() {
  const res = http.get(BASE + '/api/catalog/products?pageSize=60', { headers: GUARD });
  if (res.status !== 200) throw new Error(`seed failed: ${res.status} ${res.body}`);
  const body = res.json();
  return {
    productIds: body.items.map((i) => i.id),
    categories: body.categoryFacets.map((f) => f.key),
    manufacturers: body.manufacturerFacets.map((f) => f.key),
    totalPages: body.totalPages,
  };
}
```

8.4 LT-02 — the main load scenario

```
// load/load.js
import { group } from 'k6';
import { get, post, think, pick, catalogueSeed, SEARCH_TERMS } from './lib/common.js';

export const options = {
  insecureSkipTLSVerify: true,          // self-signed cert — NP-QA-TC-001 §2.5
  scenarios: {
    storefront: {
      executor: 'ramping-vus',
      startVUs: 0,
      stages: [
        { duration: '2m',  target: 10 },   // ramp to the measured knee
        { duration: '15m', target: 10 },   // hold — this is the measured window
        { duration: '3m',  target: 0  },   // ramp down
      ],
      gracefulRampDown: '30s',
    },
  },
  thresholds: {
    // §5.1 — whole mix
    'http_req_failed':   ['rate<0.001'],
    'np_errors':         ['rate<0.001'],
    'np_guard_rejected': ['rate==0'],      // a missing guard header fails the run outright
    'http_req_duration': ['p(50)<50', 'p(95)<400', 'p(99)<1000'],

    // §5.2 — per-endpoint budgets
    'http_req_duration{endpoint:products}':   ['p(95)<500'],
    'http_req_duration{endpoint:product}':    ['p(95)<300'],
    'http_req_duration{endpoint:suggest}':    ['p(95)<250'],
    'http_req_duration{endpoint:home}':       ['p(95)<400'],
    'http_req_duration{endpoint:content}':    ['p(95)<200'],
    'http_req_duration{endpoint:config}':     ['p(95)<150'],
    'http_req_duration{endpoint:categories}': ['p(95)<150'],
    'http_req_duration{endpoint:me}':         ['p(95)<100'],
  },
};

export function setup() { return catalogueSeed(); }

/* Five calls, exactly as the SPA issues them on a cold landing. */
function landing() {
  group('landing', () => {
    get('/api/store/config',        'config');
    get('/api/store/content',       'content');
    get('/api/catalog/categories',  'categories');
    get('/api/catalog/home',        'home');
    get('/api/auth/me',             'me');
  });
}

function browse(d) {
  group('browse', () => {
    const page = 1 + Math.floor(Math.random() * Math.min(d.totalPages, 30));
    get(`/api/catalog/products?page=${page}&pageSize=24`, 'products');
    think(8, 20);

    get(`/api/catalog/products?cat=${pick(d.categories)}&page=1&pageSize=24`, 'products');
    think(8, 20);

    get(`/api/catalog/products/${pick(d.productIds)}`, 'product');
    think(10, 40);                                  // reading the product
  });
}

function search(d) {
```

```
  group('search', () => {
    const term = pick(SEARCH_TERMS);
    // Type-ahead fires per keystroke after a debounce — model that, do not skip it.
    for (let n = 3; n <= term.length && n <= 6; n++) {
      get(`/api/catalog/suggest?q=${encodeURIComponent(term.slice(0, n))}`, 'suggest');
      think(0.15, 0.4);
    }
    get(`/api/catalog/products?q=${encodeURIComponent(term)}&page=1&pageSize=24`, 'products');
    think(8, 20);
    get(`/api/catalog/products/${pick(d.productIds)}`, 'product');
    think(10, 40);
  });
}

/* Pricing without writing anything — the closest safe proxy for checkout cost. */
function quote(d) {
  group('quote', () => {
    post('/api/cart/quote', {
      items: [{ productId: pick(d.productIds), quantity: 1 + Math.floor(Math.random() * 3) }],
      district: pick(['Dhaka', 'Chittagong', 'Sylhet', 'Rangpur']),
    }, 'quote');
    think(5, 15);
  });
}

export default function (d) {
  const roll = Math.random();
  if (roll < 0.60)      { landing(); browse(d); }        // anonymous browser — 60%
  else if (roll < 0.80) { landing(); search(d); }        // searcher          — 20%
  else                  { landing(); browse(d); quote(d); }  // shopper proxy  — 20%
  think(5, 15);
}
```

The authenticated share of the mix is represented here by `quote`, which exercises the full pricing calculation without a session and without writing. The real signed-in journey is `s3-shopper.js` and is held back until section 7.1 is resolved — running it against one shared account would produce a confident number about nothing.

8.5 LT-06 — breakpoint, open model

```
// load/breakpoint.js
import { get, pick, catalogueSeed, SEARCH_TERMS } from './lib/common.js';

export const options = {
  insecureSkipTLSVerify: true,
  scenarios: {
    find_the_cliff: {
      executor: 'ramping-arrival-rate',   // fixed ARRIVAL rate: the server cannot slow us down
      startRate: 50,
      timeUnit: '1s',
      preAllocatedVUs: 100,
      maxVUs: 1000,                       // must be free to grow as latency rises
      stages: [
        { duration: '4m', target: 150 },
        { duration: '4m', target: 250 },
        { duration: '4m', target: 350 },
        { duration: '4m', target: 500 },
        { duration: '4m', target: 600 },
      ],
    },
  },
  thresholds: {
    // abortOnFail stops the run the moment the platform is past its limit,
    // so the breaking point is recorded without flooding a shared host.
    'http_req_duration': [{ threshold: 'p(95)<1000', abortOnFail: true, delayAbortEval: '30s' }],
    'http_req_failed':   [{ threshold: 'rate<0.05',  abortOnFail: true, delayAbortEval: '30s' }],
  },
};

export function setup() { return catalogueSeed(); }

export default function (d) {
  const roll = Math.random();
  if (roll < 0.45)      get('/api/catalog/products?page=1&pageSize=24', 'products');
  else if (roll < 0.70) get(`/api/catalog/products/${pick(d.productIds)}`, 'product');
  else if (roll < 0.85) get(`/api/catalog/suggest?q=${pick(SEARCH_TERMS).slice(0, 4)}`, 'suggest');
  else                  get('/api/catalog/home', 'home');
}
```

The distinction matters. The closed VU model of LT-02 self-throttles — when responses slow, each VU issues fewer requests, so the measured load falls just as the platform struggles, and the cliff never appears. An open arrival-rate model keeps pushing, which is what a real crowd does.

8.6 LT-04 — spike

```
// load/spike.js — stages only; the body is the same as load.js
stages: [
  { duration: '3m',  target: 2  },   // calm baseline, so recovery has something to return to
  { duration: '30s', target: 80 },   // the campaign SMS goes out
  { duration: '3m',  target: 80 },   // the crowd stays
  { duration: '30s', target: 2  },   // it leaves
  { duration: '5m',  target: 2  },   // RECOVERY — the part that actually matters
],
```

The final five minutes are the test. Compare p95 over the last two minutes against the first three: if it has not returned to baseline, the platform has a queue it cannot drain, and that is a more serious finding than anything observed during the peak itself.

8.7 LT-S3 — authenticated shopper needs §7.1

```
// load/s3-shopper.js
import http from 'k6/http';
import { SharedArray } from 'k6/data';
import { get, post, think, pick, BASE, GUARD, catalogueSeed } from './lib/common.js';

/* One row per seeded account: { phone, password }. Parsed once, shared by all VUs. */
const USERS = new SharedArray('users', () => JSON.parse(open('./data/lt-users.json')));

export const options = {
  insecureSkipTLSVerify: true,
  scenarios: {
    shoppers: {
      executor: 'ramping-vus',
      stages: [{ duration: '1m', target: 10 }, { duration: '10m', target: 10 },
               { duration: '1m', target: 0 }],
      gracefulRampDown: '30s',
    },
  },
  thresholds: {
    'http_req_failed':   ['rate<0.001'],
    'np_throttled_429':  ['rate<0.01'],     // sign-in throttling must not distort the run
    'http_req_duration{endpoint:cart}':    ['p(95)<300'],
    'http_req_duration{endpoint:quote}':   ['p(95)<400'],
    'http_req_duration{endpoint:summary}': ['p(95)<400'],
  },
};

export function setup() { return catalogueSeed(); }

/* Sign in ONCE per VU and keep the cookie. Logging in every iteration would
   measure the throttle, not the platform — see §7.2. */
let jar = null;
function session() {
  if (jar) return;
  const u = USERS[(__VU - 1) % USERS.length];           // one account per VU
  const res = http.post(BASE + '/api/auth/login', JSON.stringify({
    phone: u.phone, password: u.password, rememberMe: true,
  }), { headers: { ...GUARD, 'Content-Type': 'application/json' }, tags: { endpoint: 'login' } });
  if (res.status !== 200) throw new Error(`VU ${__VU} login failed: ${res.status}`);
  jar = true;                                           // k6 keeps cookies per VU automatically
}

export default function (d) {
  session();
  get('/api/cart',             'cart');
  get('/api/wishlist/ids',     'wishlist');
  think(2, 6);

  get('/api/catalog/products?page=1&pageSize=24', 'products');
  think(8, 20);

  const productId = pick(d.productIds);
  get(`/api/catalog/products/${productId}`, 'product');
  think(10, 30);

  post('/api/cart/quote', { items: [{ productId, quantity: 2 }], district: 'Dhaka' }, 'quote');
  get('/api/account/summary', 'summary');
  think(10, 25);
}
```

`__VU` is k6's per-virtual-user index, so account *n* belongs to VU *n* and no two VUs share a cart. With fewer accounts than VUs the modulo wraps and the collisions of section 7.1 return — seed at least as many accounts as the highest VU count any scenario uses.

8.8 Running

```
k6 run load/smoke.js                                  # always first
k6 run load/load.js
k6 run --out json=load/results/$(date +%F-%H%M)-load.json load/load.js
BASE_URL=https://staging.example.com k6 run load/load.js

k6 run load/stress.js
k6 run load/spike.js
k6 run --duration 8h load/soak.js                     # overnight
k6 run load/breakpoint.js                             # agreed window only

k6 run load/s1-catalogue.js                           # targeted scenarios
k6 run load/s2-suggest.js
```

8.9 Continuous integration

Run LT-01 on every build as a fast guard, and LT-02 nightly against a quiet environment. Do not put LT-02 on every pull request: a twenty-minute test on a shared host is both slow and unreliable as a merge gate, and a noisy gate quickly becomes an ignored one.

```
load-smoke:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: grafana/setup-k6-action@v1
    - run: k6 run load/smoke.js
      env:
        BASE_URL: ${{ secrets.BASE_URL }}

load-nightly:
  if: github.event_name == 'schedule'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: grafana/setup-k6-action@v1
    - run: k6 run --out json=summary.json load/load.js
      env:
        BASE_URL: ${{ secrets.BASE_URL }}
    - uses: actions/upload-artifact@v4
      if: ${{ !cancelled() }}
      with: { name: k6-summary, path: summary.json }
```

k6 exits non-zero when a threshold is breached, so the job fails on its own — no result parsing is needed to make the gate work.

9 Monitoring and analysis

9.1 Client-side metrics

k6 reports these without configuration. The four that decide a run are marked.

METRIC	WHAT IT TELLS YOU
`http_req_duration` ★	End-to-end latency. Always read p95 and p99, never the mean
`http_reqs` ★	Throughput. Compare against the ~190 RPS ceiling
`http_req_failed` ★	Share of non-2xx
`np_guard_rejected` ★	Custom. Anything above zero means the script is broken, not the platform
`http_req_waiting`	Time to first byte — server think time, excluding transfer
`http_req_connecting` / `tls_handshaking`	Should be near zero after warm-up. If not, connections are not being reused
`http_req_receiving`	Transfer time. Rises with payload size — the metric that would show compression working
`data_received`	Total bytes. Divide by `http_reqs` for the real average payload
`vus` and `iterations`	Confirms the profile ran as intended and the generator kept up
`np_throttled_429`	Custom. Rate limiting engaged — information, not necessarily failure

9.2 Server-side metrics

A load test that only watches the client tells you *that* something is slow, never *why*. Capture these on the host for the duration of every run.

LAYER	METRIC	INTERPRETATION
Host	CPU, overall and per core	Saturated CPU with flat throughput means the application is the limit
	Memory, working set	Steady growth during a soak is a leak
	Disk queue length	Usually logging, occasionally the database
IIS	Requests queued, requests per second	A growing queue with flat CPU means a thread or connection limit, not capacity
	Current connections	Should track VUs; divergence means connection churn
	Application-pool recycles	Any recycle during a run invalidates it
ASP.NET Core	Thread-pool queue length	The usual cause of a sudden latency cliff
	GC pause time and gen-2 collections	Explains p99 spikes that p50 hides
	Exceptions per second	Should be flat at zero

LAYER	METRIC	INTERPRETATION
SQL Server	Batch requests per second	Compare with API RPS — a ratio far above 1 suggests an N+1 query
	Average wait time, top wait types	Names the actual bottleneck
	Slowest queries by total duration	Expect `catalog/products` at the top; confirm it

> **The one correlation worth making every time**
>
> Put client p95 and server CPU on the same timeline. If latency climbs while CPU stays moderate, the limit is a queue, a lock or a pool — and more hardware will not help. If they climb together, the limit is real work, and either the work gets cheaper or the host gets bigger. The section 3 ramp — flat throughput, linearly rising latency, zero errors — has the shape of the first case, which is worth confirming on the host before anyone buys anything.

9.3 Reading a result

1. **Check `np_guard_rejected` is zero.** If it is not, stop — the run measured the rejection path and every other number is meaningless.
2. **Check the profile ran.** `vus_max` and iteration count should match the intended shape; if the generator could not keep up, it is the bottleneck.
3. **Read the error rate before the latency.** Fast failures look like excellent latency.
4. **Read p95 and p99 together.** A gap between them that widens with load is a queueing or GC signature.
5. **Compare throughput against offered load.** Flat throughput with rising latency is saturation; falling throughput is something worse.
6. **Break down by endpoint tag.** One slow endpoint drags the aggregate and is usually the whole story.
7. **Compare against the previous run,** not only against the threshold. A run that passes at 390 ms after passing at 180 ms last month is a problem that has not reported itself yet.

9.4 Common misreadings

LOOKS LIKE	USUALLY IS
Excellent latency, enormous throughput	The guard header is missing and every request is a 400 from middleware
Latency rises the moment the test starts	No think time — the VUs are a benchmark loop, not users
Throughput stops rising with more VUs	Either real saturation, or the generator is out of CPU. Check the generator first
First iteration far slower than the rest	Cold start, JIT, connection and TLS setup. Discard a warm-up window
A clean run that passes every threshold	Check `http_reqs` — a scenario that silently failed to start also passes everything
Sudden 429s in an authenticated scenario	VUs are sharing accounts and tripping sign-in throttling — §7.1
Results differ run to run on the same build	A shared environment with other activity. Re-run in a quiet window before investigating

10 Tuning candidates

Five changes suggested by the baseline, ranked by expected effect against the work required. These are hypotheses for the development team, not conclusions — each should be measured before and after with LT-02, which is precisely what the plan exists to make easy.

T-1 Enable response compression highest value, lowest effort

Observation	No content encoding is negotiated. The catalogue page is 17.3 KB and the home payload 19.3 KB, uncompressed on the wire
Expected effect	JSON of this shape typically compresses by 80–85%. A 17.3 KB page becomes roughly 3 KB
Who benefits	Mobile customers, who are the dominant channel. On the LAN the baseline barely moves — which is exactly why this would never show up in a local test
Work	Enable Brotli and gzip in IIS. Configuration, not code
Verify	`data_received ÷ http_reqs` before and after; `http_req_receiving` p95

T-2 Cache the static configuration endpoints

Observation	Every page load fetches `store/config`, `store/content`, `catalog/categories` and `catalog/home`. Together these are **29% of all requests** in the workload model, and their content changes perhaps weekly
Expected effect	A short public `Cache-Control` on those four would remove roughly a quarter of all traffic before it reaches the application
Caution	`auth/me` must stay uncacheable — it is per-user. Only the four anonymous, identical-for-everyone endpoints qualify
Work	Add `Cache-Control: public, max-age=300` plus an `ETag`; invalidate when an administrator saves content or settings
Verify	Request count per endpoint in LT-02 should fall sharply for those four

T-3 Profile the catalogue query

Observation	`catalog/products` costs 15 ms against 1–5 ms for everything else, and returns faceted counts over 17,453 rows alongside the page itself
Hypothesis	The facet aggregation is the expensive half, and it is recomputed on every request including simple paging
Candidates	Cache facet counts per filter combination; compute facets only on the first page of a result set; add covering indexes for the common filter and sort pairs
Work	Needs a query plan first. Run LT-S1 while capturing the slowest queries
Verify	LT-S1 p95 before and after

T-4 Enable HTTP/2

Observation	HTTP/1.1 only. A landing page issues five API calls plus assets, against a six-connection-per-origin browser limit

Expected effect	Multiplexing removes head-of-line blocking across those five calls and reduces connection setup. Most visible on high-latency mobile links
Work	IIS configuration; requires TLS, which is already in place
Caution	Will not change the server-side numbers in section 3 at all — the gain is in the browser. Measure it with browser timings, not with k6

T-5 Paginate the order list

Observation	`GET /api/orders` returns every order a customer has ever placed — the only unpaginated collection in the API. Finding F-5 in NP-API-TS-001
Expected effect	Bounded cost per request regardless of customer history. Today the most loyal customers are the most expensive to serve, which is the wrong way round
Work	Add the standard envelope with a default page size; keep the first page shape compatible for existing clients
Verify	LT-S6 with a 500-order customer, before and after

10.6 Summary

RANK	CHANGE	EFFORT	EXPECTED GAIN	WHO NOTICES
1	T-1 Compression	Very low	80–85% fewer bytes	Every mobile customer
2	T-2 Cache static config	Low	~29% fewer requests	The server, at peak
3	T-3 Catalogue query	Medium	Raises the ceiling	Everyone, under load
4	T-4 HTTP/2	Low	Faster page assembly	Mobile customers
5	T-5 Paginate orders	Medium	Bounded growth	Long-standing customers

T-1 and T-2 together are a few hours of configuration and would remove roughly a quarter of the requests and four fifths of the bytes. Neither requires an application change, and neither would have been identified without measuring first.

11 Execution, risks and reporting

11.1 Schedule

TEST	CADENCE	DURATION	WINDOW
LT-01 Smoke	Every build	1 min	Any
LT-02 Load	Nightly and every release candidate	20 min	Quiet hours
LT-03 Stress	Each release	30 min	Agreed window
LT-04 Spike	Each release	12 min	Agreed window
LT-05 Soak	Monthly and before major releases	2–8 h	Overnight
LT-06 Breakpoint	Quarterly, or after any architecture change	Up to 20 min	Agreed window, supervised
LT-S1…S6	As needed, and after any related tuning	10–15 min each	Quiet hours

11.2 Entry and exit criteria

ENTRY	EXIT
Functional suite green — a load test on a broken build measures nothing	LT-01 to LT-04 executed with results recorded
Environment quiet and the window agreed	All section 5.1 SLOs met at target load
Server-side monitoring in place and capturing	Degradation mode characterised in writing
LT-01 passes	No unexplained regression against the previous run
Generator has spare CPU at the intended VU count	Every WARN and FAIL has an owner and a date

11.3 Risks

RISK	LIKELIHOOD	IMPACT	MITIGATION
Shared environment produces noisy results	High	Medium	Agreed windows; repeat any surprising run before acting on it
One account blocks authenticated scenarios	Certain	High	Seed 100 accounts — §7.1. Until then run the read scenarios and say so
Generator is the bottleneck, not the host	Medium	High	Watch generator CPU; distribute k6 if needed; treat ceilings as lower bounds
LT-06 disrupts other work	Medium	Medium	Supervised window, `abortOnFail` thresholds, announce before starting
Test environment is not production-sized	High	High	Report ratios and shapes rather than absolute capacity; re-baseline on production-like hardware before sizing decisions

RISK	LIKELIHOOD	IMPACT	MITIGATION
Results quoted without the section 3.5 caveats	Medium	Medium	Every report carries the caveats; no figure leaves the team without them

11.4 Report template

```
North Pharmacy — Load Test Report
Run id        LT-02 / 2026-11-03 / 02:00 Asia-Dhaka
Build         commit abc1234
Environment   https://202.4.101.118:1165
Scenario      LT-02 Load — 10 VU, 15 min hold
Generator     1 host, peak CPU 34%          (not the bottleneck)

RESULT        PASS / WARN / FAIL

Throughput    ___ RPS      (ceiling ~190, target >= 180)
p50           ___ ms       (target < 50)
p95           ___ ms       (target < 400)
p99           ___ ms       (target < 1000)
Errors        ___ %        (target < 0.1)
Guard rejects 0            (must be 0 or the run is void)

Per endpoint        p95      budget   verdict
  catalog/products  ___ ms   500 ms   ___
  catalog/product   ___ ms   300 ms   ___
  catalog/suggest   ___ ms   250 ms   ___
  catalog/home      ___ ms   400 ms   ___
  store/config      ___ ms   150 ms   ___

Server            peak   mean
  CPU             ___%   ___%
  IIS queue       ___    ___
  SQL batches/s   ___    ___
  GC gen-2        ___

Versus previous run   p95 ___% ; throughput ___%
Observations          ...
Actions               owner / date
Caveats               §3.5 applies
```

12 Appendices

A Endpoints used by the load scenarios

All reads, all safe, none rate limited. The full API surface is documented in NP-API-TS-001 sections 4 and 5.

METHOD	PATH	TAG	USED BY
GET	`/api/store/config`	`config`	Every journey
GET	`/api/store/content`	`content`	Every journey
GET	`/api/catalog/categories`	`categories`	Every journey
GET	`/api/catalog/home`	`home`	Landing
GET	`/api/auth/me`	`me`	Every journey
GET	`/api/catalog/products`	`products`	Browse, search, LT-S1
GET	`/api/catalog/products/{id}`	`product`	Browse, search
GET	`/api/catalog/suggest`	`suggest`	Search, LT-S2
POST	`/api/cart/quote`	`quote`	Shopper — writes nothing
POST	`/api/auth/login`	`login`	LT-S3, LT-S4 — once per VU only
GET	`/api/cart`	`cart`	LT-S3
GET	`/api/account/summary`	`summary`	LT-S3
GET	`/api/orders`	`orders`	LT-S6
GET	`/api/admin/dashboard`	`dashboard`	LT-S4
GET	`/api/admin/orders`	`adm_orders`	LT-S4
GET	`/api/admin/inventory`	`adm_inv`	LT-S4

B Account seeding requirement

The format expected by `load/data/lt-users.json`:

```
[
  { "phone": "01900000001", "password": "LoadTest@2026" },
  { "phone": "01900000002", "password": "LoadTest@2026" },
  ...
  { "phone": "01900000100", "password": "LoadTest@2026" }
]
```

REQUIREMENT	DETAIL
Count	100 customers, 5 administrators — at least as many as the highest VU count any scenario uses
Numbers	A reserved block that cannot collide with real customers
Password	One shared value is fine; these are load accounts, not security fixtures

REQUIREMENT	DETAIL
State	Empty cart and wishlist at the start of each run
History	One account seeded with 500+ orders for LT-S6
Reseed	A script that resets them, runnable before any load window

Creating these through `POST /api/auth/register` is not viable — the endpoint rate limits at roughly ten per minute, so a hundred accounts would take ten minutes of careful pacing and leave no way to reset them. They should be seeded directly.

C Reproducing the baseline

The section 3 figures came from two short Node scripts: a sequential sampler and a concurrency ramp, both using `https.Agent({ keepAlive: true, rejectUnauthorized: false })` and sending the guard header. They are kept in `load/baseline/` so the measurement can be repeated exactly. Once k6 is installed, `load/smoke.js` and `load/load.js` supersede them — the originals remain only so that the numbers quoted in this document can be reproduced as they were taken.

D Glossary extension

TERM	MEANING
Closed model	Fixed number of VUs, each waiting for a response before the next request. Self-throttling
Open model	Fixed arrival rate regardless of response time. Models a real crowd; used by LT-06
Little's Law	Concurrency = throughput × latency. Used in §3.2 to derive mean in-flight time
Head-of-line blocking	On HTTP/1.1, one slow response blocks others on the same connection
Warm-up	The first requests of a run, inflated by JIT, connection setup and cold caches. Excluded from analysis
Guard rejection	A 400 caused by the missing `X-Requested-With` header. Voids a run

E Revision history

VER	DATE	AUTHOR	CHANGE
1.0	25 Oct 2026	QA Automation	First issue. Baseline measured; 6 test types and 6 targeted scenarios defined; k6 implementation supplied; 5 tuning candidates raised

> **The first thing to do with this plan**
>
> Not to run LT-06 and find the cliff — that number is interesting but not actionable today. It is to seed the hundred accounts of section 7.1, so the authenticated half of the platform stops being unmeasurable, and to enable compression and static caching from section 10, which together are a few hours of configuration and remove roughly a quarter of the requests and four fifths of the bytes. Then re-run LT-02 and watch the baseline move.